

Python for Data Science

Buổi 6: Pandas Nâng Cao II – Data Cleaning, GroupBy & Pivot Table

PGS-TS Nguyễn Mạnh Hùng

August 19, 2026

Cấu trúc Buổi 6

- 1 Ôn tập Buổi 3–5 và đặt vấn đề
- 2 Module 21: Làm sạch dữ liệu I – Trùng lặp & Thay thế giá trị
- 3 Module 22: Làm sạch dữ liệu II – Phân nhóm, Outlier & Mã hóa
- 4 Module 23: GroupBy – Split-Apply-Combine
- 5 Module 24: Pivot Table
- 6 Bài tập áp dụng: MovieLens 1M, Titanic

Lưu ý phạm vi

Missing data, MultiIndex và merge/join đã học ở Buổi 5. Buổi 6 xử lý các dạng "bẩn" còn lại của dữ liệu thực tế (trùng lặp, ngoại lệ) và học cách **tổng hợp** dữ liệu theo nhóm – hai kỹ năng bắt buộc trước khi bước sang Trực quan hóa (Buổi 8–9).

Nhắc lại Buổi 3–5

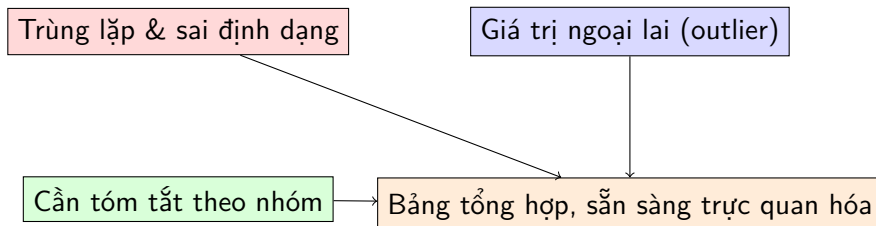
- Buổi 3 – Series/DataFrame: ndarray gắn nhãn; `.loc[]`/`.iloc[]`, boolean indexing.
- Buổi 4 – đọc dữ liệu từ nhiều định dạng: CSV nâng cao, JSON, HTML, XML, Excel, SQL...
- Buổi 5 – `isnull/dropna/fillna` (dữ liệu thiếu), `MultiIndex`, `concat`, `merge/join` (kết hợp nhiều bảng).

Hai vấn đề còn bỏ ngỏ

- ❶ Dữ liệu thiếu (NaN) không phải dạng "bẩn" duy nhất – còn **trùng lặp**, **sai định dạng**, **giá trị ngoại lai (outlier)**.
- ❷ Có một bảng sạch rồi thì làm sao **tóm tắt** nó theo từng nhóm (ví dụ: theo giới tính, theo hạng vé, theo phương pháp phát hiện)?

Buổi 6 trả lời trực tiếp cả hai câu hỏi này.

Dữ liệu thực tế: bẩn và rời rạc



Data cleaning (Module 21–22) loại bỏ nhiễu ở mức *từng dòng*; GroupBy & Pivot Table (Module 23–24) tổng hợp dữ liệu ở mức *từng nhóm* – hai bước bắt buộc trước khi trực quan hóa hoặc mô hình hóa.

Module 21: Làm sạch dữ liệu I – Trùng lặp & Thay thế giá trị

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Phát hiện và loại bỏ dòng trùng lặp bằng `duplicated()/drop_duplicates()`.
- 2 Ánh xạ giá trị theo danh mục bằng `map()`.
- 3 Thay thế giá trị sentinel (mã hóa dữ liệu thiếu kiểu cũ) bằng `replace()`.
- 4 Đổi tên nhãn hàng/cột bằng `rename()`.

Phát hiện & loại bỏ dòng trùng lặp

```
data = pd.DataFrame({"k1": ["one","two"]*3 + ["two"], "k2": [1,1,2,3,3,4,4]})
print(data.duplicated())      # True ở dòng lặp lại HOÀN TOÀN với 1 dòng trước đó
print(data.drop_duplicates()) # bỏ các dòng trùng lặp

data["v1"] = range(7)
print(data.drop_duplicates(subset=["k1"]))      # chỉ xét trùng lặp theo cột k1
print(data.drop_duplicates(["k1","k2"], keep="last")) # giữ dòng CUỐI CÙNG thay vì đầu
```

`duplicated()` mặc định so khớp *toàn bộ* các cột; tham số `subset` thu hẹp phạm vi so khớp, `keep="last"/"first"/False` quyết định dòng nào được giữ lại (nguồn: notebook PDA ch07 – Data Cleaning and Preparation).

Ánh xạ giá trị theo danh mục: map()

```
data = pd.DataFrame({
    "food": ["bacon", "pulled pork", "bacon", "pastrami",
            "corned beef", "bacon", "pastrami", "honey ham", "nova lox"],
    "ounces": [4, 3, 12, 6, 7.5, 8, 3, 5, 6]
})

meat_to_animal = {"bacon": "pig", "pulled pork": "pig", "pastrami": "cow",
                  "corned beef": "cow", "honey ham": "pig", "nova lox": "salmon"}

data["animal"] = data["food"].map(meat_to_animal)  # tạo cột mới từ từ điển ánh xạ
print(data)
```

map() là công cụ tiêu chuẩn để chuyển một cột giá trị "thô" (tên món ăn) thành một cột giá trị "đã diễn giải" (loại động vật) dựa trên một từ điển hoặc hàm tùy ý – rất hữu ích khi chuẩn hóa dữ liệu phân loại.

Thay thế giá trị sentinel: replace()

```
data = pd.Series([1., -999., 2., -999., -1000., 3.])
print(data.replace(-999, np.nan))           # thay 1 gia tri
print(data.replace([-999, -1000], np.nan))   # thay nhieu gia tri bang 1 gia tri
print(data.replace([-999, -1000], [np.nan, 0])) # thay tuong ung tung cap
print(data.replace({-999: np.nan, -1000: 0})) # cach viet gon bang dict
```

Vì sao cần replace()?

Nhiều hệ thống cũ dùng số "ma thuật" (ví dụ -999, -1000) để mã hóa dữ liệu thiếu thay vì NaN chuẩn. `replace()` chuyển các sentinel này về NaN để có thể áp dụng `isnull()/dropna()/fillna()` đã học ở Buổi 5.

Đổi tên nhãn: rename()

```
data = pd.DataFrame(np.arange(12).reshape((3, 4)),
                    index=["Ohio", "Colorado", "New York"],
                    columns=["one", "two", "three", "four"])

print(data.rename(index=str.title, columns=str.upper))
#           ONE  TWO  THREE  FOUR
# Ohio         0    1     2     3
# Colorado     4    5     6     7
# New York     8    9    10    11

print(data.rename(index={"Ohio": "INDIANA"}, columns={"three": "peekaboo"}))
```

`rename()` nhận một *hàm* (áp dụng cho mọi nhãn, ví dụ `str.title/str.upper`) hoặc một *dict* (chỉ đổi tên các nhãn được liệt kê) – không sửa DataFrame gốc trừ khi truyền `inplace=True`.

Tổng kết Module 21

- `uplicated()/drop_duplicates()`: phát hiện và loại bỏ dòng trùng lặp (chú ý `subset, keep`).
- `map()`: ánh xạ giá trị theo dict hoặc hàm – chuẩn hóa dữ liệu phân loại.
- `replace()`: chuyển sentinel (`-999...`) về `NaN` chuẩn hoặc giá trị hợp lệ.
- `rename()`: đổi tên nhãn hàng/cột bằng hàm hoặc dict.

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Phân nhóm dữ liệu liên tục thành các khoảng rời rạc bằng `pd.cut()`.
- 2 Phân nhóm theo phân vị bằng `pd.qcut()`.
- 3 Phát hiện và xử lý giá trị ngoại lai (outlier).
- 4 Mã hóa biến phân loại thành biến giả (dummy variables) bằng `pd.get_dummies()`.

Phân nhóm dữ liệu liên tục: `pd.cut()`

```
ages = [20, 22, 25, 27, 21, 23, 37, 31, 61, 45, 41, 32]
bins = [18, 25, 35, 60, 100]
age_categories = pd.cut(ages, bins)      # (18,25], (25,35], (35,60], (60,100]
print(age_categories.codes)              # chỉ số nhóm: [0 0 0 1 0 0 2 1 3 2 2 1]
print(pd.Series(age_categories).value_counts())

group_names = ["Youth", "YoungAdult", "MiddleAged", "Senior"]
print(pd.cut(ages, bins, labels=group_names))  # đặt nhãn để đọc thay vì khoảng số
```

`pd.cut()` chuyển một cột số liên tục (tuổi, thu nhập...) thành các *khoảng rời rạc* có ranh giới cố định do người dùng chọn – bước chuẩn bị phổ biến trước khi `groupby()` hoặc `pivot_table()` theo nhóm tuổi.

Phân nhóm theo phân vị: `pd.qcut()`

```
rng = np.random.RandomState(12345)
data = rng.standard_normal(1000)
quartiles = pd.qcut(data, 4, precision=2)    # chia thành 4 phân vị (mỗi nhóm ~25%)
print(pd.Series(quartiles).value_counts())
# (-2.96, -0.68]      250
# (-0.68, -0.01]     250
# (-0.01, 0.63]      250
# (0.63, 3.93]       250

pd.qcut(data, [0, 0.1, 0.5, 0.9, 1.])      # các mốc phân vị tùy chỉnh
```

Khác với `cut()` (ranh giới *cố định* do người dùng chọn), `qcut()` chọn ranh giới sao cho *số lượng phần tử mỗi nhóm gần bằng nhau* – phù hợp khi dữ liệu phân bố lệch và muốn nhóm đều nhau về kích thước thay vì đều nhau về độ rộng khoảng.

Phát hiện & xử lý giá trị ngoại lai (outlier)

```
rng = np.random.RandomState(12345)
data = pd.DataFrame(rng.standard_normal((1000, 4)))

col = data[2]
print((col.abs() > 3).sum())          # 4 <- so gia tri "bat thuong" o cot 2
print(data[(data.abs() > 3).any(axis="columns")].shape)  # (10, 4) <- 10 dong co outlier

data[data.abs() > 3] = np.sign(data) * 3      # "cat ngon" (cap) ve +-3
print(data.describe().round(3))              # max/min gio day dung bang +-3.0
```

Kỹ thuật *capping*: thay vì xóa hẳn dòng chứa outlier (mất dữ liệu), ta giới hạn giá trị về một ngưỡng hợp lý (ở đây ± 3 , tương ứng $\pm 3\sigma$ với dữ liệu chuẩn hóa) bằng `np.sign(data) * 3` – giữ được số dòng nhưng loại bỏ ảnh hưởng cực đoan.

Mã hóa biến phân loại: `pd.get_dummies()`

```
df = pd.DataFrame({"key": ["b","b","a","c","a","b"], "data1": range(6)})
print(pd.get_dummies(df["key"]))
#           a         b         c
# 0  False     True  False
# 1  False     True  False
# 2   True    False  False
# ...
dummies = pd.get_dummies(df["key"], prefix="key")
df_with_dummy = df[["data1"]].join(dummies)    # ghép lại với cột dữ liệu gốc
```

`get_dummies()` chuyển 1 cột phân loại có k giá trị khác nhau thành k cột nhị phân (kỹ thuật "one-hot encoding") – bước bắt buộc trước khi đưa dữ liệu phân loại vào phần lớn thuật toán Machine Learning (sẽ học từ Buổi 10).

Ứng dụng get_dummies() với dữ liệu multi-label thật

```
mnames = ["movie_id", "title", "genres"]
movies = pd.read_table("datasets/movielens/movies.dat", sep="::", header=None,
                      names=mnames, engine="python", encoding="latin-1")

print(movies.head())
```

#	movie_id	title	genres
# 0	1	Toy Story (1995)	Animation Children's Comedy
# 1	2	Jumanji (1995)	Adventure Children's Fantasy

```
dummies = movies["genres"].str.get_dummies("|") # 1 phim co THE co nhieu the loai
print(dummies.iloc[:5, :6]) # (3883, 18) -- 18 cot the loai nhi phan
```

`Series.str.get_dummies(sep)` xử lý trường hợp *multi-label* (1 phim thuộc nhiều thể loại cùng lúc, phân tách bởi "|") – không thể dùng `get_dummies()` thông thường (nguồn: kho *python-for-data-analysis*, `datasets/movielens/movies.dat`, 3883 phim thật).

Tổng kết Module 22

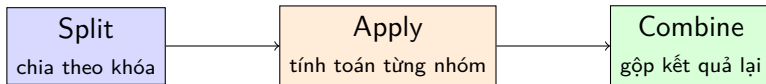
- `pd.cut(x, bins)`: phân nhóm theo ranh giới cố định; `labels=` đặt tên nhóm.
- `pd.qcut(x, q)`: phân nhóm theo phân vị – số phần tử mỗi nhóm gần bằng nhau.
- Outlier: phát hiện bằng ngưỡng (`.abs() > k`), xử lý bằng capping (`np.sign(x)*k`) hoặc loại bỏ.
- `pd.get_dummies()`: one-hot encoding; `Series.str.get_dummies(sep)` cho dữ liệu multi-label.

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Giải thích mô hình Split-Apply-Combine đằng sau `groupby()`.
- 2 Sử dụng `aggregate()`, `filter()`, `transform()`, `apply()` trên đối tượng `GroupBy`.
- 3 Chỉ định khóa nhóm theo nhiều cách khác nhau (cột, dict, hàm).
- 4 Áp dụng `groupby()` để phân tích một tập dữ liệu thực tế nhiều chiều.

Ý tưởng Split-Apply-Combine



- **Split:** chia DataFrame thành các nhóm con theo giá trị của 1 cột (hoặc nhiều cột).
- **Apply:** tính một hàm (tổng hợp, biến đổi, hoặc lọc) *độc lập* trên từng nhóm.
- **Combine:** gộp kết quả của từng nhóm trở lại thành 1 cấu trúc dữ liệu duy nhất.

Tên gọi "group by" bắt nguồn từ SQL; ý tưởng chia-tính-gộp này là nền tảng của hầu hết các thao tác tổng hợp dữ liệu.

groupby() cơ bản: lazy evaluation

```
df = pd.DataFrame({"key": ["A","B","C","A","B","C"], "data": range(6)},
                  columns=["key", "data"])

grouped = df.groupby("key")      # KHÔNG tính toán gì cả -- chỉ là 1 "kế hoạch"
print(type(grouped))             # DataFrameGroupBy

print(grouped.sum())             # phép tính CHI xảy ra khi gọi hàm tổng hợp
#      data
# key
# A         3
# B         5
# C         7
```

`df.groupby("key")` trả về một đối tượng `DataFrameGroupBy` – một "kế hoạch chia nhóm" chưa thực sự tính toán gì (lazy evaluation). Việc tính toán chỉ xảy ra khi ta gọi một hàm tổng hợp (`.sum()`, `.mean()`...) lên đối tượng này.

Ứng dụng thật: dữ liệu Ngoại hành tinh (Planets)

```
import seaborn as sns
planets = sns.load_dataset("planets")      # 1035 ngoại hành tinh, dữ liệu thật
print(planets.shape, planets.columns.tolist())

print(planets.groupby("method")["orbital_period"].median())
# method
# Astrometry          631.180000
# Radial Velocity      360.200000
# Transit              5.714932
# ...
```

Chỉ mục cột (["orbital_period"]) sau groupby() chọn 1 cột cụ thể để tổng hợp, tránh tính toán thừa trên các cột không cần thiết. *Lưu ý:* sns.load_dataset() tải dữ liệu qua Internet (và lưu cache cục bộ) – cần kết nối mạng ở lần chạy đầu tiên.

aggregate(): nhiều hàm tổng hợp cùng lúc

```
rng = np.random.RandomState(0)
df = pd.DataFrame({"key": ["A","B","C","A","B","C"],
                    "data1": range(6), "data2": rng.randint(0, 10, 6)},
                  columns=["key", "data1", "data2"])

print(df.groupby("key").aggregate(["min", "median", "max"]))    # 3 ham cho MOI cot

print(df.groupby("key").aggregate({"data1": "min", "data2": "max"})) # ham rieng tung cot
#      data1  data2
# key
# A         0      5
# B         1      7
# C         2      9
```

aggregate() (hay viết tắt agg()) nhận một chuỗi tên hàm, một hàm, một danh sách hàm, hoặc một dict ánh xạ cột → hàm – linh hoạt hơn nhiều so với gọi trực tiếp .sum()/ .mean().

filter(): lọc theo tính chất của cả nhóm

```
def filter_func(x):  
    return x["data2"].std() > 4  
  
print(df.groupby("key").std(numeric_only=True))  
#           data1      data2  
# key  
# A      2.12132  1.414214    <- do lech chuan data2 của nhóm A không > 4  
# B      2.12132  4.949747  
# C      2.12132  4.242641  
  
print(df.groupby("key").filter(filter_func))    # nhóm A bị loại vì không thỏa điều kiện
```

`filter()` nhận một hàm trả về True/False cho *mỗi nhóm*; toàn bộ dòng thuộc nhóm bị đánh giá False sẽ bị loại khỏi kết quả – khác với `aggregate()` (thu gọn) và `transform()` (giữ nguyên hình dạng, trang sau).

transform(): biến đổi giữ nguyên hình dạng

```
def center(x):  
    return x - x.mean()          # trừ đi trung bình của CHÍNH NHÓM do  
  
print(df.groupby("key")[["data1", "data2"]].transform(center))  
#    data1  data2  
# 0   -1.5    1.0  
# 1   -1.5   -3.5  
# 2   -1.5   -3.0  
# ...      (van con 6 dong, dung bang df goc -- khong bi thu gon)
```

transform() trả về kết quả **cùng hình dạng** với dữ liệu gốc (không thu gọn như aggregate()) – ứng dụng kinh điển: chuẩn hóa dữ liệu theo từng nhóm (trừ đi trung bình nhóm, chia cho độ lệch chuẩn nhóm) để so sánh các quan sát công bằng giữa các nhóm.

apply(): hàm tùy ý trên từng nhóm

```
def norm_by_data2(x):  
    # x là 1 DataFrame ung voi 1 nhóm  
    x = x.copy()  
    x["data1"] /= x["data2"].sum()  
    return x  
  
print(df.groupby("key").apply(norm_by_data2, include_groups=False))
```

apply() là công cụ *tổng quát nhất*: hàm truyền vào nhận một DataFrame (đại diện 1 nhóm) và có thể trả về bất kỳ đối tượng Pandas nào (Series, DataFrame, hoặc giá trị vô hướng) – Pandas tự động quyết định cách gộp kết quả dựa trên kiểu trả về. Linh hoạt nhưng thường *chậm hơn* aggregate()/transform() có sẵn.

Nhiều cách chỉ định khóa nhóm

1) Mot list/array cung do dai voi DataFrame

```
L = [0, 1, 0, 1, 2, 0]
```

```
df.groupby(L).sum()
```

2) Dict hoac Series anh xa TU INDEX sang nhom

```
df2 = df.set_index("key")
```

```
mapping = {"A": "vowel", "B": "consonant", "C": "consonant"}
```

```
df2.groupby(mapping).sum()
```

3) Ham bat ky (nhan gia tri index, tra ve nhan nhom)

```
df2.groupby(str.lower).mean(numeric_only=True)
```

4) Ket hop nhieu cach -> nhom theo NHIEU cap (MultiIndex)

```
df2.groupby([str.lower, mapping]).mean(numeric_only=True)
```

Ví dụ tổng hợp: ngoại hành tinh theo phương pháp & thập kỷ

```
decade = 10 * (planets["year"] // 10)
decade = decade.astype(str) + "s"
decade.name = "decade"

result = planets.groupby(["method", decade])["number"].sum().unstack().fillna(0)
print(result)
```

#	1980s	1990s	2000s	2010s
# Radial Velocity	1.0	52.0	475.0	424.0
# Transit	0.0	0.0	64.0	712.0
# Microlensing	0.0	0.0	12.0	15.0
# ...				

Chỉ vài dòng code kết hợp `groupby()` nhiều khóa (phương pháp + thập kỷ tạo từ Series tùy biến), `unstack()` (Buổi 5) và `fillna()` (Buổi 5) đã cho thấy bức tranh toàn cảnh: phương pháp Transit chỉ thực sự bùng nổ từ những năm 2010 (nhờ vệ tinh Kepler).

Tổng kết Module 23

- `groupby()`: lazy evaluation – chỉ tính khi gọi hàm tổng hợp.
- `aggregate()`: 1 hoặc nhiều hàm, cho toàn bộ cột hoặc theo dict từng cột.
- `filter()`: giữ/loại *cả nhóm* theo điều kiện; `transform()`: biến đổi giữ nguyên hình dạng.
- `apply()`: hàm tùy ý, tổng quát nhất nhưng thường chậm hơn.
- Khóa nhóm có thể là tên cột, list, dict, hàm, hoặc kết hợp nhiều loại (nhóm nhiều cấp).

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Giải thích mối liên hệ giữa `pivot_table()` và `groupby()` nhiều cấp.
- 2 Xây dựng pivot table nhiều cấp (dùng `pd.cut()` làm chỉ mục).
- 3 Sử dụng tham số `margins` để tính tổng theo lề bảng.
- 4 Áp dụng `pivot_table()` trên dữ liệu thực tế (`births.csv`).

Động lực: từ groupby thủ công đến pivot table

```
import seaborn as sns
titanic = sns.load_dataset("titanic")    # 891 hành khách tàu Titanic, dữ liệu thật

print(titanic.groupby("sex")[["survived"]].mean())
#           survived
# female    0.742038
# male      0.188908

# Muốn xem theo CA giới tính VÀ hạng vé -> code bắt đầu "roi rác":
print(titanic.groupby(["sex", "class"])[["survived"]].aggregate("mean").unstack())
```

Kết quả đúng nhưng chuỗi groupby → aggregate → unstack khá dài dòng cho một thao tác 2 chiều rất phổ biến – đây chính xác là điều pivot_table() giải quyết.

Cú pháp pivot_table()

```
print(titanic.pivot_table("survived", index="sex", columns="class", aggfunc="mean"))  
# class      First      Second      Third  
# sex  
# female  0.968085  0.921053  0.500000  
# male    0.368852  0.157407  0.135447
```

Cùng kết quả với trang trước nhưng dễ đọc hơn nhiều. Nữ hạng nhất sống sót gần như chắc chắn (96,8%); nam hạng ba sống sót thấp nhất (13,5%). Cú pháp đầy đủ:

```
pivot_table(values, index, columns, aggfunc="mean", fill_value=None,  
margins=False).
```

Pivot table nhiều cấp: `pd.cut()` làm chỉ mục

```
age = pd.cut(titanic["age"], [0, 18, 80])      # 2 nhóm tuổi: (0,18] và (18,80]
print(titanic.pivot_table("survived", ["sex", age], "class"))
```

# class		First	Second	Third
# sex	age			
# female	(0, 18]	0.909091	1.000000	0.511628
#	(18, 80]	0.972973	0.900000	0.423729
# male	(0, 18]	0.800000	0.600000	0.215686
#	(18, 80]	0.375000	0.071429	0.133663

Truyền một *danh sách* (`["sex", age]`) làm index tạo pivot table với chỉ mục hàng nhiều cấp – kết quả tương ứng `MultiIndex` đã học ở Buổi 5. Có thể làm tương tự cho columns (ví dụ thêm `pd.qcut()` theo giá vé).

Tham số margins: tổng theo lề bảng

```
print(titanic.pivot_table("survived", index="sex", columns="class", margins=True))  
# class      First      Second      Third      All  
# sex  
# female  0.968085  0.921053  0.500000  0.742038  
# male    0.368852  0.157407  0.135447  0.188908  
# All     0.629630  0.472826  0.242363  0.383838
```

`margins=True` thêm hàng/cột "All": tỉ lệ sống sót chung theo giới tính (bỏ qua hạng vé), theo hạng vé (bỏ qua giới tính), và tỉ lệ sống sót chung toàn bộ tàu (38,4%) – tất cả trong 1 lệnh duy nhất.

Ứng dụng thật: births.csv theo thập kỷ & giới tính

```
births = pd.read_csv("notebooks/data/births.csv")    # kho PythonDataScienceHandbook
births["decade"] = 10 * (births["year"] // 10)
```

```
print(births.pivot_table("births", index="decade", columns="gender", aggfunc="sum"))
```

# gender	F	M
# decade		
# 1960	1753634	1846572
# 1970	16263075	17121550
# 1980	18310351	19243452
# 1990	19479454	20420553
# 2000	18229309	19106428

Dữ liệu số ca sinh thật tại Hoa Kỳ (CDC): số bé trai sinh ra luôn nhiều hơn số bé gái ở *mọi* thập kỷ – một hiện tượng nhân khẩu học đã biết (tỉ số giới tính khi sinh ≈ 105 nam/100 nữ).

Tổng kết Module 24

- `pivot_table() = groupby()` nhiều cấp + `unstack()`, viết gọn trong 1 lệnh.
- `index/columns` có thể nhận danh sách (kể cả kết quả `pd.cut()/pd.qcut()`) để tạo bảng nhiều cấp.
- `aggfunc`: hàm tổng hợp (mặc định "mean"), có thể là dict riêng cho từng cột giá trị.
- `margins=True`: thêm tổng theo lề (hàng/cột "All").

Ví dụ đầy đủ: phim được nữ giới đánh giá cao nhất (MovieLens 1M)

```
unames = ["user_id", "gender", "age", "occupation", "zip"]
users = pd.read_table("datasets/movielens/users.dat", sep="::", header=None,
                     names=unames, engine="python", encoding="latin-1")
rnames = ["user_id", "movie_id", "rating", "timestamp"]
ratings = pd.read_table("datasets/movielens/ratings.dat", sep="::", header=None,
                       names=rnames, engine="python", encoding="latin-1")
data = pd.merge(pd.merge(ratings, users), movies)    # merge 3 bang (Buoi 5)

mean_ratings = data.pivot_table("rating", index="title", columns="gender",
                                aggfunc="mean")
ratings_by_title = data.groupby("title").size()
active_titles = ratings_by_title.index[ratings_by_title >= 250]    # loc phim it luot danh gia
print(mean_ratings.loc[active_titles].sort_values("F", ascending=False).head(3))
```

Kết quả kiểm chứng thực tế (1 000 209 lượt đánh giá, 3 706 phim, 1 216 phim đủ ≥ 250 lượt đánh giá): phim được nữ giới đánh giá cao nhất là *Close Shave, A (1995)* (F: 4,64 / M: 4,47), tiếp theo *Wrong Trousers, The (1993)* và *Sunset Blvd. (1950)*.

Bài tập 1: Mở rộng phân tích MovieLens

Dữ liệu (kho *python-for-data-analysis*, `datasets/movielens/`: `users.dat`, `ratings.dat`, `movies.dat`). Tự chạy lại pipeline `merge + pivot_table` ở trang trước, sau đó mở rộng:

- 1 Trong số phim "active" (≥ 250 lượt đánh giá), tìm 5 phim được **nam giới** đánh giá cao nhất – có trùng với danh sách của nữ giới không?
- 2 Tính cột `diff = mean_ratings["M"] - mean_ratings["F"]`, sắp xếp tăng dần – đây là các phim *nữ giới thích hơn nam giới rõ rệt nhất*. (Gợi ý kiểm chứng: *Dirty Dancing (1987)* nên đứng đầu danh sách này.)
- 3 Dùng `movies["genres"].str.get_dummies("|")` (Module 22) kết hợp `groupby` để tính rating trung bình theo **từng thể loại phim**.
- 4 Dùng `pd.cut()` chia cột `age` của bảng `users` thành các nhóm tuổi, rồi lập pivot table rating trung bình theo nhóm tuổi $\times$ giới tính.

Bài tập 2: Làm sạch & tổng hợp trên Titanic

Dữ liệu: `seaborn.load_dataset("titanic")` (891 hành khách).

- ❶ Kiểm tra `titanic.duplicated().sum()` – có dòng trùng lặp hoàn toàn không? Kiểm tra thêm `isnull().sum()` theo từng cột (Buổi 5) – cột nào thiếu dữ liệu nhiều nhất?
- ❷ Dùng `pd.qcut(titanic["fare"], 4)` chia giá vé thành 4 nhóm; lập pivot table tỉ lệ sống sót theo nhóm giá vé $\times$ hạng vé (`class`).
- ❸ Dùng `groupby(["embark_town", "class"])["survived"].agg(["mean", "count"])` – cảng lên tàu nào có tỉ lệ sống sót cao nhất? Số lượng hành khách có đủ lớn để kết luận đáng tin cậy không?
- ❹ *Câu hỏi thảo luận:* vì sao nên xem đồng thời **mean** và **count** khi so sánh tỉ lệ giữa các nhóm, thay vì chỉ nhìn mean?

Bảng tổng hợp các thao tác Pandas trong Buổi 6

Nhóm	Hàm/thao tác	Mục đích
Trùng lặp	<code>uplicated</code> , <code>drop_duplicates</code>	Phát hiện, loại bỏ dòng trùng lặp
Ảnh xạ/thay thế	<code>map</code> , <code>replace</code> , <code>rename</code>	Chuẩn hóa giá trị, sentinel, nhãn
Phân nhóm	<code>pd.cut</code> , <code>pd.qcut</code>	Rời rạc hóa dữ liệu liên tục
Outlier	<code>.abs() > k</code> , <code>np.sign(x)*k</code>	Phát hiện, capping giá trị ngoại lai
Mã hóa	<code>pd.get_dummies</code> , <code>.str.get_dummies</code>	One-hot encoding (đơn/multi-label)
Tổng hợp	<code>groupby (agg, filter, transform, apply)</code>	Split-Apply-Combine theo nhóm
Pivot	<code>pivot_table (aggfunc, margins)</code>	Tổng hợp 2+ chiều, thay cho groupby lồng nhau

Tài liệu tham khảo I

- VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.
- McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 51–56.
- VanderPlas, J. (n.d.). *PythonDataScienceHandbook* [Kho mã nguồn – notebooks 03.08 (Aggregation and Grouping), 03.09 (Pivot Tables); dữ liệu `births.csv`]. GitHub.
<https://github.com/jakevdp/PythonDataScienceHandbook>
- Pugh, D. R. (n.d.). *python-for-data-analysis* [Kho mã nguồn – notebook ch07 (Data Cleaning and Preparation); dữ liệu `movielens`]. GitHub.
<https://github.com/davidrpugh/python-for-data-analysis>
- GroupLens Research. *MovieLens 1M Dataset*. University of Minnesota.
<https://grouplens.org/datasets/movielens/1m/>
- Waskom, M. L. (2021). seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60), 3021. (nguồn dữ liệu `planets`, `titanic` qua `seaborn.load_dataset()`)

- The pandas development team. (2020). *pandas-dev/pandas: Pandas (Version 1.0) [Software]*. Zenodo. <https://doi.org/10.5281/zenodo.3509134>

Chuẩn bị cho buổi tiếp theo: Time Series & Pandas Hiệu Năng Cao

Thông điệp

Data cleaning, GroupBy và Pivot Table hoàn thiện bộ kỹ năng "làm sạch – tổng hợp" dữ liệu bảng. Buổi 7 mở rộng sang chiều *thời gian* (time series, resample, rolling window) và các kỹ thuật tối ưu hiệu năng (`eval()`/`query()`) – bước chuẩn bị cuối cùng trước khi bước sang Trực quan hóa dữ liệu (Buổi 8–9).